电子科技大学 自动化工程 学院

标 准 实 验 报 告

（实验）课程名称 通信与网络技术

电子科技大学教务处制表

电 子 科 技 大 学

实 验 报 告

学生姓名：刘超然　　学 号：2023060904025　　指导教师：马文建

实验地点：主楼 C2-511　　实验时间：10 月 25 日

一、实验室名称

现代测试技术实验基地

二、实验项目名称

基于 MATLAB/Simulink 的链路层差错控制编码分析

三、实验学时

4 学时

四、实验原理

CRC(Cyclic Redundancy Check, 循环冗余校验码)是一种广泛用于检错的循环码，广泛应用于移动通信系统中，用于实现自动请求重传(ARQ)功能。CRC 是利用线性编码理论，在发送端根据需要传送的 k 位二进制信息码序列 $C(x)$，以一定的规则产生一个检验用的 r 位 CRC 码 $R(x)$，附加在信息码后面，构成一个新的共 $n=k+r$ 位的二进制传送码 $T(x)$，此种编码方式称为(n,k)码。对于一个给定的(n,k)码，可以证明存在一个最高次幂为 r 的生成多项式 $G(x)$，根据生成多项式 $G(x)$对 k 位信息码进行运算产生 r 位校验码。

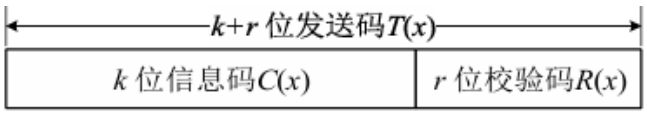


图2-1　CRC校验码构成

CRC 编解码规则如下：

(1)将发送信号 $C(x)$左移 r 位，空出 r 位即为校正码的位置，将 $C(x)\times 2^r$ 除以生成多项式 $G(x)$得到的余数即为校验码。如图 2-2 所示，发送的二进制序列为 10110011，生成多项式 $G(x) = x^4+x^3+1$，二进制比特为 11001，生成多项式 $G(x)$为 5 位，则校验码为 5-1=4 位，所以在发送码后加 4 个 0，得到 101100110000，用“模 2 除法（即异或运算）”得到的余数 0100 即为 CRC 校验码，最终发送的二进制码为 101100110100。

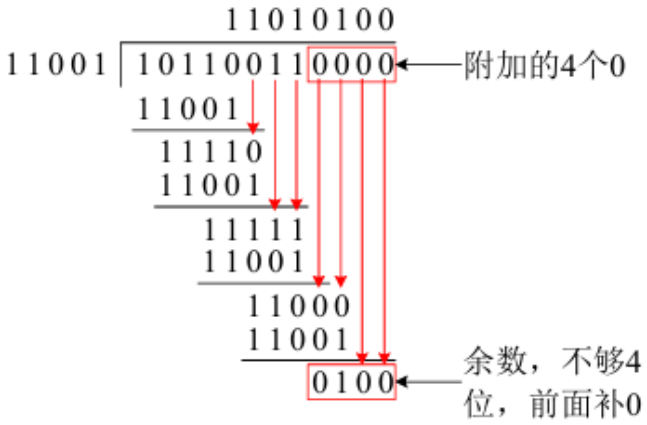


图2-2　CRC校验码计算示例

(2)接收端将收到的二进制序列除以生成多项式 $G(x)$，如果余数为 0，则说明传输中无错误发生，否则传输有误。同样以图 2-2 中的举例，接收到的 101100110100 除以生成多项式 11001，余数为 0，则无差错。

常用的生成多项式如表 2-1 所示

表2-1 常用的生成多项式$G(x)$

名称	生成多项式$G(x)$	二进制码	数值式
CRC-4	x^4+x+1	10011	0x13
CRC-8	$x^8+x^5+x^4+1$	100110001	0x131
CRC-8	x^8+x^2+x+1	100000111	0x107
CRC-12	$x^{12}+x^{11}+x^3+x^2+x+1$	1100000001111	0x180F
CRC-16	$x^{16}+x^{15}+x^2+1$	1100000000000101	0x18005
CRC-32	$x^{32}+x^{26}+x^{23}+x^{22}+x^{16}+x^{12}+x^{11}+x^{10}+x^8+x^7+x^5+x^4+x^2+x+1$	1000001001100000 10001110110110111	0x104C11DB7

五、实验目的

理解 CRC 校验码的校验原理，掌握 CRC 编解码方法，通过 MATLAB 和 Simulink 仿真对比分析 CRC 校验码的校验性能。

六、实验内容

1) CRC 编解码过程
2) CRC 链路 MATLAB 仿真
3) CRC 链路 Simulink 仿真

七、实验器材（设备、元器件）

计算机 MATLAB 环境

八、实验步骤

（一）CRC 编解码过程

使用 MATLAB 仿真产生一个 16bit 随机数，采用 CRC-4 冗余校验方式(多项式为 x^4+x+1)，实现 CRC 编码和解码过程，并手动加个错误 bit，查看校验信息。

MATLAB 代码

```
clear;
poly_g = [1 0 0 1 1];  % 生成多项式G(x)=x^4+x+1，CRC-4
len_r = length(poly_g) - 1;  % CRC码的长度：生成多项式长度减1
len_data = 16;  % 消息比特数
msg_init = randi(2,1,len_data)-1;  % 消息比特
msg_temp = [msg_init zeros(1,len_r)];  % 消息比特左移
[q, r] = deconv(msg_temp, poly_g);  % 多项式除法求CRC码，q为商，r为余数

crc_code = mod(r(end - len_r + 1: end), 2);  % 取余数的最后len_r位模2运算
msg_tx = [msg_init crc_code];  % 发送帧：原始数据 + 校验冗余码
%% 解码验证：将接收到的16+R位编码和生成多项式相除
[q, r] = deconv(msg_tx, poly_g);
msg_rx = mod(r(end - len_r + 1: end), 2);
disp(msg_rx)    %如果正确，得到余数全是0
%% 检错验证：手动传错
msg_tx_err = msg_tx;
msg_tx_err(2) = 1 - msg_tx_err(2);  % 假设第二位出现错误
[q, r] = deconv(msg_tx_err,poly_g);
msg_rx = mod(r(end - len_r + 1: end), 2);
disp(msg_rx)
```



（二）CRC 链路 MATLAB 仿真

使用 MATLAB 仿真 CRC-8 校验码在二进制对称信道中的检错性能。其中每帧的消息比特个数为 16，二进制对称信道采用 16QAM 调制，E_b/N_0 的范围为 0~10dB。

MATLAB 代码

```
clear;
N = 100000;  % 发送的帧数
len_data = 16;  % 一帧中的消息比特个数
poly_g = [1 0 0 1 1 0 0 0 1];  % CRC-8生成多项式
len_r = length(poly_g) - 1;  % CRC码的长度
EbNo = 0:2:10;  %SNR范围
ber = berawgn(EbNo, 'qam', 16);  % 16-QAM理论误比特率
for indx = 1:length(ber)
    pe = ber(indx);  % BSC信道错误概率
    err = zeros(1,N);  % 传输的帧是否产生误码
    err1 = zeros(1,N);  % 通过CRC检测传输的帧是否产生误码
    for iter = 1:N
        msg = randi(2, 1, len_data)-1;  % 消息比特
        msg1 = [msg zeros(1, len_r)];  % 消息比特左移
        [q, r] = deconv(msg1, poly_g);  % 用多项式除法求CRC校验码,q为商，r为余数
        r = mod(abs(r), 2);  % 进行模2处理
        crc = r(len_data+1:end);  % CRC校验码
        frame = [msg crc];  % 发送帧
        x = bsc(frame, pe);  % 通过二进制对称信道：0到1和1到0错误概率相同
        [q1, r1] = deconv(x, poly_g);  % 接收序列除以多项式
        r1 = mod(abs(r1), 2);  % 模 2 处理
        err(iter) = biterr(frame, x);  % 统计本帧是否产生误码
        err1(iter) = sum(r1);  % 通过 CRC 统计本帧是否产生误码
    end
    fer1(indx) = sum(err~=0);  % 误帧率
    fer2(indx) = sum(err1~=0);  % 通过 CRC 计算误帧率
end
pmissed = (fer1 - fer2)/N;  % CRC 漏检的概率
semilogy(EbNo, pmissed, '-ko');
title('MATLAB CRC-8 检错性能');
xlabel('Eb/No'); ylabel('漏检概率');
grid on;
```

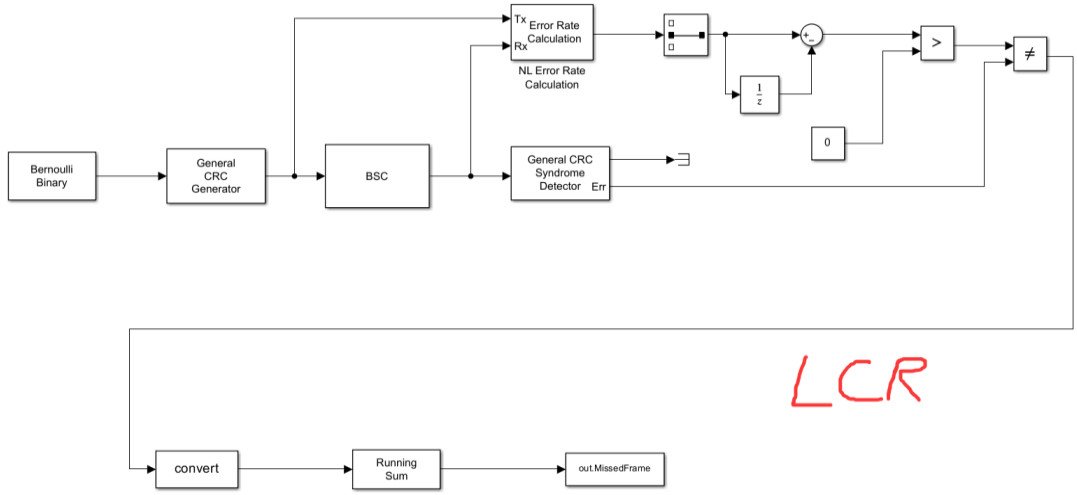
（三）CRC 链路 Simulink 仿真

系统模块框图如图所示，产生的二进制信号经过 Binary Symmetric Channel 后分为两路，其中一路与 CRC 编码后的数据帧一起通过 Error Rate Calculation 模块进行比较，得到的结果通过 Selector 模块选择第 2 个输出信号，即误比特的个数作为输出。

为判断本帧是否出现错误传输，把 Selector 模块输出信号与它一个单位延迟信号相减，如果差值为 0，则说明本帧的误比特数没有增加，即本帧没有错误；如果差值不等于 0，则表示本帧出现错误。通过 Relational Operator 与 0 比较得到判别结果。经过 CRC-N Syndrome Detector 后有两个输出信号，第 1 个输出端口为除去 CRC 的信息序列，第 2 个输出端口表示接收信号的 CRC 校验结果，如果信息位重新计算得到的 CRC 与接收到的 CRC 相等，则输出信号等于 0；否则输出信号等于 1。

将 CRC 检测器与误码率计算模块的输出结果进行比较，如果结果相同，则说明 CRC 校验正确，否则说明 CRC 校正发生错误判决。Cumulative Sum 模块对 CRC 校正发生错误判决的次数进行统计，通过 To Workspace 模块传到 MATLAB

工作区。



16-QAM 调制下的 CRC-8 Simulink 模型

九、实验数据及结果分析

（一）CRC 编解码过程

运行代码得到如下结果：

```
>> L3_CSC_1
     0     0     0     0

     1     0     0     0
```

结果：正常传输接收校验结果：0 0 0 0

出错传输接收校验结果：1 0 0 0

len_data 修改为 256，运行结果：

```
>> L3_CSC_1
     0     0     0     0

     0     0     0     0
```

分析与结论：

1. **复现分析：**在 len_data=16 时，可以看出，正常解码得到的校验结果全为 0，表示无误码；而手动在第 2 位加入错误后，解码得到的余数不为 0，说明 CRC 成功检测出了错误。

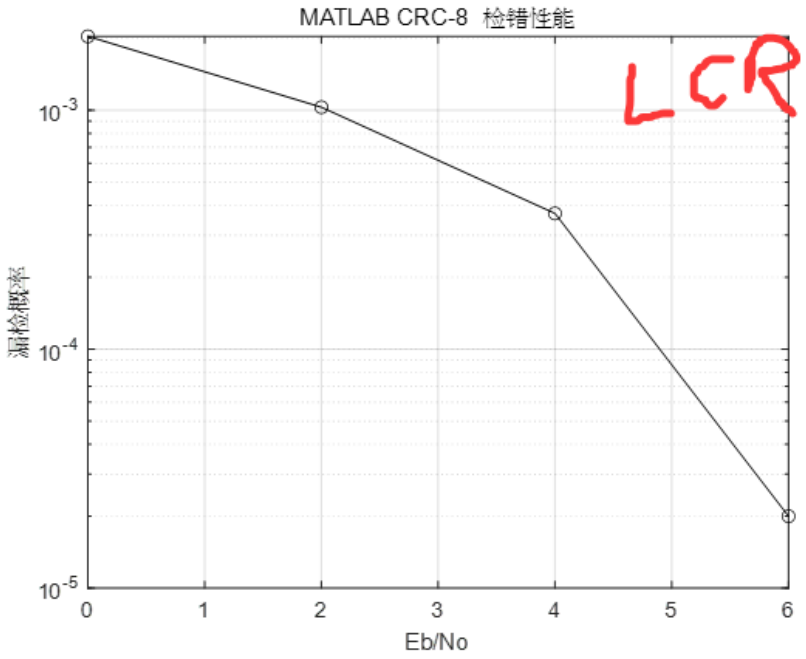
2. **长数据分析：**将 len_data 修改为 256 后，仿真结果显示的余数往往全为 0(即显示无错误)，即使我们人为加入了错误。

原因：这主要是因为 MATLAB 中的 deconv 函数是基于浮点数运算的。当数

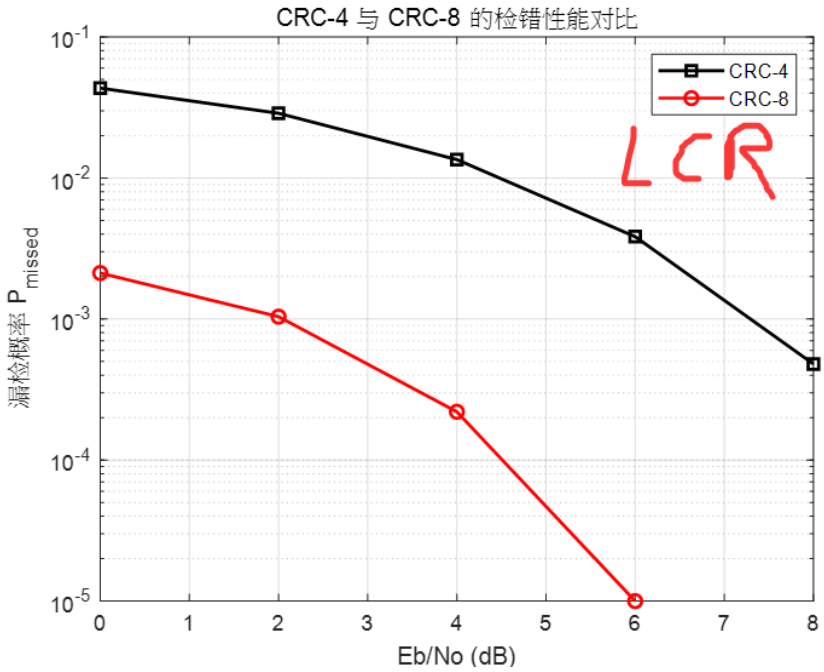
据长度（多项式阶数）过大时（如 256 位），会产生数值精度误差，导致模 2 运算后的余数计算不准确。此外，CRC-4 的检错能力有限，对于过长的数据帧，其漏检概率也会增加。

改进建议：若要进行长数据的精确 CRC 校验，建议使用 MATLAB 的 Communications Toolbox 中的专用 CRC 对象或者自行编写基于模 2 运算的循环算法，而不是直接使用 deconv。

（二）CRC 链路 MATLAB 仿真



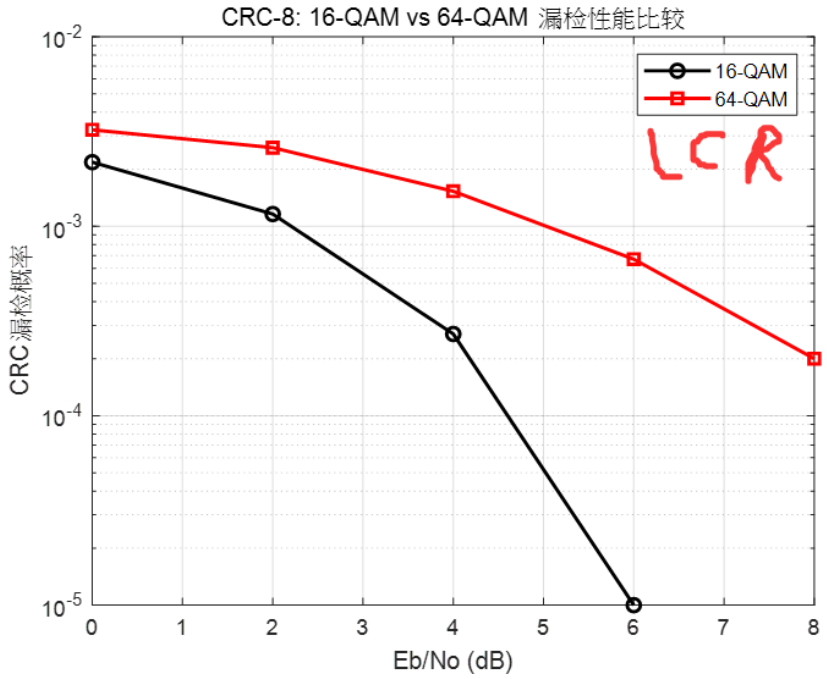
Simulink 16-QAM 调制下的 CRC-8 检错性能



CRC-4 和 CRC-8 的检错性能对比

分析与结论：如图所示，在相同的信噪比（E_b/N_0）条件下，CRC-8 的漏检概率明显低于 CRC-4。

原因：生成多项式的阶数越高（r 值越大），校验位的长度越长，其检错能力越强，漏检概率越低。CRC-8 提供了 8 位冗余校验，比 CRC-4 能更有效地覆盖错误模式。



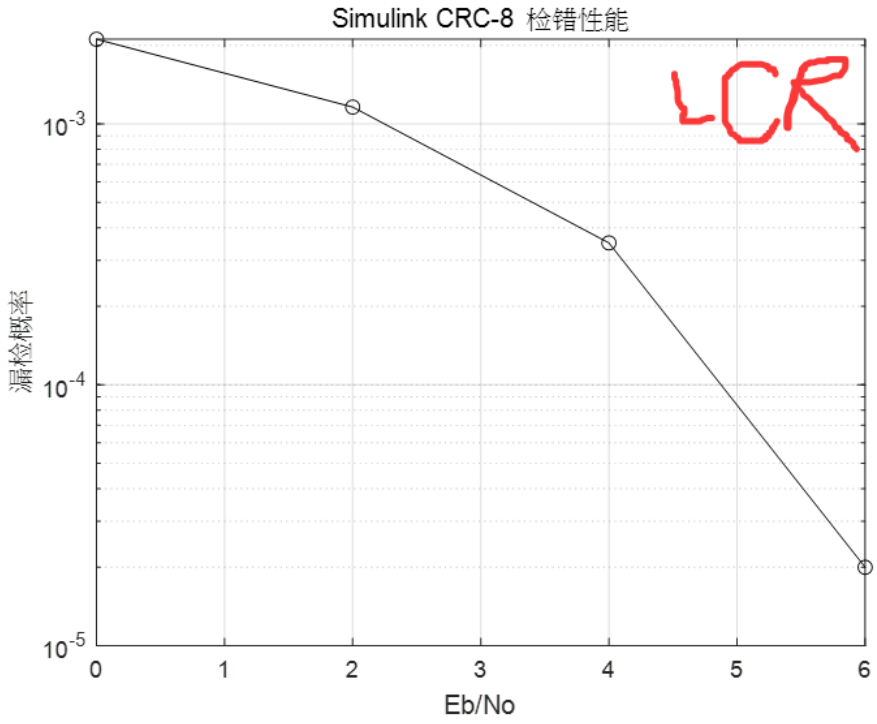
16-QAM 和 64-QAM 在同一校验码下的检错性能对比

分析与结论：如图所示，在使用相同的 CRC 校验码（如 CRC-8）时，64-QAM 调制方式下的漏检概率高于 16-QAM。

原因：64-QAM 的星座点更密集，在相同的信噪比下，符号间的欧氏距离更小，更容易受到噪声干扰而发生误判，导致误比特率更高。误比特率的增加会导致一帧中出现更多错误比特，超出了 CRC 的检错能力范围，从而导致漏检率上升。

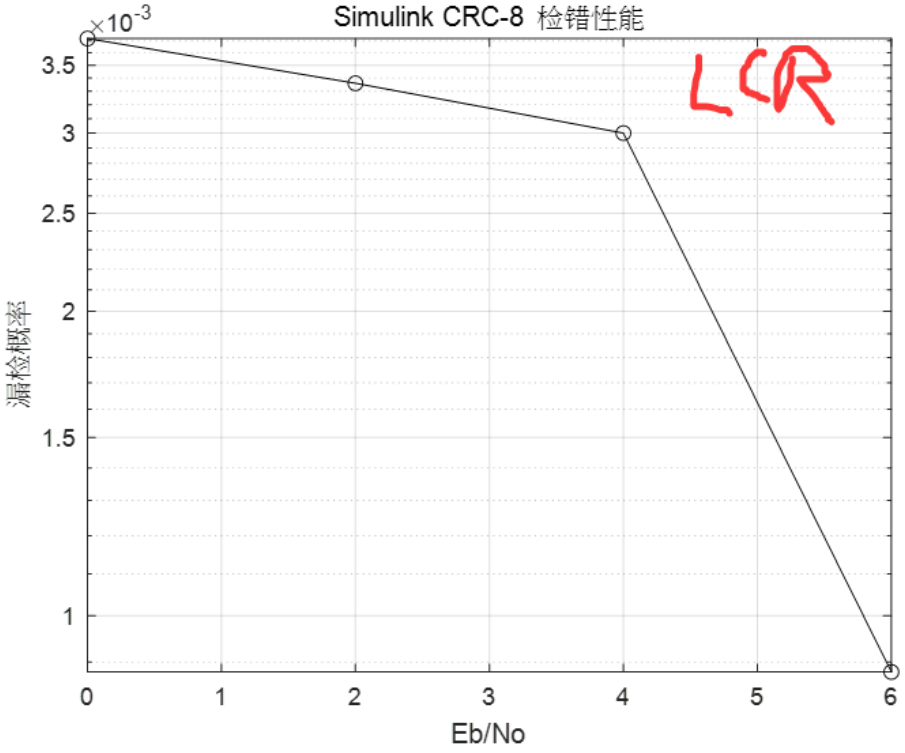
（三）CRC 链路 Simulink 仿真

```
clear;
EbNo = 0:2:10; % SNR 的范围
BIT=16;
ber = berawgn(EbNo,'qam',16);
pmissed = zeros(1, length(EbNo));
for ii = 1:length(EbNo) % 依次仿真不同 EbN0 下的 BER
 BER = ber(ii); % 赋值给 BSC 信道模块中的 BER
 out = sim('L3_Sim_CSC_1.slx');     % 返回 SimulationOutput 对象
 MissedFrame = out.MissedFrame; % 从对象中取出信号
 pmissed(ii) = MissedFrame(end)/length(MissedFrame);
end
semilogy(EbNo, pmissed, '-ko');
title('Simulink CRC-8 检错性能')
xlabel('Eb/No');ylabel('漏检概率')
grid on;
```



Simulink 16-QAM 调制下的 CRC-8 检错性能 16bit

```
clear;
BIT= 64;    % ← 每帧消息比特数从 16 改为 64
EbNo = 0:2:10; % SNR 范围
ber = berawgn(EbNo,'qam',16);
pmissed = zeros(1, length(EbNo));
for ii = 1:length(EbNo)
    BER = ber(ii);    % 给 BSC 信道用
    out = sim('L3_Sim_CSC_1.slx');  % 返回 SimulationOutput
    MissedFrame = out.MissedFrame;
    pmissed(ii) = MissedFrame(end)/length(MissedFrame);
end
semilogy(EbNo, pmissed, '-ko');
title('Simulink CRC-8 检错性能')
xlabel('Eb/No'); ylabel('漏检概率')
grid on;
```



Simulink 16-QAM 调制下的 CRC-8 检错性能 64bit

分析与结论：将消息比特数从 16 修改为 64 后，在相同的 CRC-8 校验下，64bit 帧长的漏检概率通常会高于 16bit 帧长。

原因：在误比特率一定的情况下，帧长越长，该帧中出现错误比特的概率就越高，且出现多位错误（超出 CRC 检测能力）的概率也随之增加。

改进：在此基础上，如果将校验码升级为 CRC-16，可以观察到漏检概率显著下降。这是因为 CRC-16 具有更长的校验位，能够检测出更多类型的错误模式（如突发错误），从而弥补了帧长增加带来的可靠性下降问题。

十、实验结论

1. **CRC 原理验证：**实验验证了 CRC 校验的基本原理，即发送端生成余数，接收端验证余数是否为 0。正常传输时余数为 0，出错时余数不为 0，实现了差错检测功能。

2. **阶数对性能的影响：**通过对比 CRC-4 和 CRC-8，证实了生成多项式阶数越高（校验位越长），系统的检错能力越强，漏检概率越低。

3. **调制方式的影响：**在相同信噪比下，高阶调制（如 64-QAM）由于抗噪声性能较差，误码率较高，导致 CRC 的漏检概率也相应升高。

4. **帧长的影响：**数据帧越长，在传输过程中出错的概率越大，对校验码的检错能力要求也越高。增加校验位长度（如从 CRC-8 到 CRC-16）可以有效改善长帧传输的可靠性。

十一、总结及心得体会

这次实验让我认识到，虽然 CRC 是一种高效的检错手段，但它并非万能。仿真结果显示，在信道环境较差（低 SNR）或帧长过长时，仍然存在漏检的可能性。这让我理解了通信系统设计中“可靠性”与“有效性”的权衡——我们不能无限增加校验位来追求零误码，而应根据实际信道条件选择合适阶数的生成多项式。此外，通过对比 MATLAB 代码和 Simulink 模块的实现方式，我进一步掌握了使用可视化工具进行通信系统建模的方法，提高了解决实际工程问题的能力。

导入思考题：

(1) 数据在传输过程中为什么会出现误码？哪些情况下会出现误码？

(2) 使用校验码后是否就一定能检查出数据的错误？

答：(1) 数据传输出现误码的主要原因包括：

噪声干扰：传输信道中存在热噪声、散粒噪声等加性高斯白噪声。

信号衰减：信号在传输介质中传播时能量会随距离衰减。

多径效应：无线信号的反射、折射导致多径衰落。

外部干扰：电磁干扰、串扰等。

(2) 不一定。CRC 校验只能检测出错误，但不能保证 100%检测出所有错误。如果传输过程中发生的错误模式恰好是生成多项式的倍数（即错误图案能被 G(x) 整除），接收端计算的余数仍然为 0，从而导致“漏检”。

拓展思考题：CRC 多项式阶数越高，漏检概率越低，即校验性能越好，但

带来的结果就是数据传输效率的降低。如何权衡差错控制下的校验性能和传输效率？

答：**低误码率环境：**在信道质量较好（如光纤传输）的情况下，可以选择较短的 CRC 校验码（如 CRC-8 或 CRC-16），以减少冗余开销，提高有效数据的传输效率。

高误码率环境：在信道环境较差（如无线信道）或对数据可靠性要求极高（如银行交易）的场景下，应选择阶数较高的 CRC（如 CRC-32），虽然牺牲了一定的传输效率（增加了冗余比特），但能显著降低漏检概率，保证数据正确性。

帧长匹配：对于长数据帧，应匹配更高阶的 CRC；短数据帧则可使用低阶 CRC，以维持合理的开销比例。

十二、对本实验过程及方法、手段的改进建议：

1. 增加算法实现：目前实验主要调用 MATLAB 库函数，建议增加手动编写 CRC 模 2 除法算法的环节，以加深对底层计算过程的理解。

2. 引入真实信道数据：可以尝试导入真实的信道噪声数据或衰落模型，而不是仅使用理想的 BSC 或 AWGN 模型，使仿真更贴近实际工程。

报告评分：

指导教师签字：